Plan nadogradnje

Allocation Header

Motivacija

Trenutna implementacija koristi AllocationMap (hash tabelu sa 64 segmenta) da prati mapiranje ptr → heapId. Ovo unosi overhead:

- Hesiranje pokazivaca pri svakoj alokaciji i dealokaciji
- Lock contention na segment lockovima AllocationMap-a
- Linear probing i upravljanje tombstone-ima
- Dodatna memorija za hash tabelu

Allocation Header eliminise sve ovo cuvanjem metapodataka direktno uz alocirani blok.

Dizajn

Svaka alokacija sadrzi mali header neposredno ispred korisnicke memorije:

```
[ AllocationHeader (8 bytes) | korisnicka memorija ]
                               ^
                               vraceno korisniku
```

Polje	Tip	Velicina	Opis
heapId	uint32_t	4 bytes	Koji heap je izvrsio alokaciju
magic	uint32_t	4 bytes	Validaciona vrednost za debug

Prednosti i nedostaci

Prednosti	Nedostaci
Eliminacija AllocationMap (nema hash tabele)	8 bytes overhead po alokaciji
O(1) dealokacija bez hesiranja	Alignment zahtijeva paznju
Nema dodatnih lockova	Teze debugovanje (nema centralne mape)
Bolja lokalnost kesa	Magic vrijednost moze dati lazne rezultate u edge casovima
Jednostavnija implementacija	

Vizualizacija rezultata

3.1 Motivacija

Trenutni benchmark ispisuje rezultate u konzolu ali ne cuva ih ni vizualizuje. Vizualizacija omogucava jasno poredenje AHM vs malloc i demonstraciju poboljsanja.

3.2 Koraci implementacije

1. Dodati CSV ispis na kraju run_ahm_benchmark() i run_malloc_benchmark()
2. Format: threads, malloc_sec, ahm_sec, speedup
3. Napisati Python skript koji ucitava CSV i generise grafik (matplotlib)
4. Grafik: X osa = broj niti, Y osa = vrijeme
5. Drugi grafik sa speedup-om po broju niti(mozda)

3.3 Primjer CSV izlaza

```
threads,malloc_sec,ahm_sec,speedup
1,2.341,2.512,0.93
2,4.102,2.891,1.42
5,7.834,2.103,3.73
10,12.441,2.044,6.09
50,38.221,3.102,12.32
```